Machine Learning
Semester 2026-III
Workshop No. 3 — Reinforcement Learning (Part B)

Eng. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Welcome to the third workshop of the *Machine Learning* course project!

Your team has developed predictive models and established strong baseline performance in earlier work. This workshop explores reinforcement learning as an alternative or complementary modeling approach.

Reinforcement learning differs fundamentally from supervised learning: instead of learning from labeled examples, an agent learns by interacting with an environment and receiving reward signals. The agent discovers strategies to maximize cumulative reward through trial and error.

This workshop covers designing and implementing an RL-based solution appropriate to your problem domain. Formulate the problem as a Markov Decision Process (`MDP`), implement a custom environment, and train an RL agent. The specific environment structure, action space, reward function, and agent algorithm should be tailored to your assigned task.

General Workshop Definition: Reinforcement learning involves formulating a problem as a sequential decision process where an agent learns through interaction with an environment by receiving rewards for actions. In this workshop, teams implement the full RL pipeline: problem formulation as an MDP, custom environment implementation, agent design with appropriate exploration and learning strategies, hyperparameter tuning, and comprehensive evaluation. The chosen RL approach should align naturally with your problem domain rather than forcing an unnatural fit.

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to Carlos A. Sierra at: *cavirguezs@udistrital.edu.co*.

Workshop Scope and Objectives

- **`MDP` Formulation:** Define and document all components of the Markov Decision Process: state space, action set, transition function, reward function, and episode termination conditions. The formulation should be grounded in your domain and leverage insights from Workshop 1 (baseline models) and Workshop 2 (deep learning).
- **Environment Implementation:** Implement a `gymnasium`-compatible environment class that simulates the RL task. The environment should accept actions from the agent and return updated states and rewards.
- **Agent Algorithm:** Implement a suitable RL algorithm for your problem. Deep Q-Networks (`DQN`) work well for discrete action spaces with Markovian dynamics. Policy gradient or actor-critic methods may be more appropriate for other settings.
- **Hyperparameter Search:** Train multiple configurations varying key hyperparameters. Compare using relevant performance metrics.
- **Evaluation:** Report comprehensive performance metrics on a held-out test set or validation environment. Compare against Workshop 1/2 baselines and domain-relevant heuristics.
- **Training Visualization:** Log all training runs and produce diagnostic visualizations showing learning curves, exploration dynamics, and sample trajectories.

Reinforcement Learning Foundations

1. **Foundational RL Papers:**
   - Sutton & Barto (2018), *Reinforcement Learning: An Introduction*, 2nd edition — comprehensive textbook covering MDPs, value functions, policy gradient, temporal difference learning. The foundation for all RL work. Free at `incompleteideas.net`.
   - Watkins & Dayan (1992), "*Q-Learning*", *Machine Learning* — the original `Q`-learning paper. Simple model-free algorithm that learns optimal action values. Basis for `DQN`.
   - Mnih et al. (2015), "*Human-level Control through Deep Reinforcement Learning*", *Nature 2015* — the original `DQN` paper combining `Q`-learning with deep neural networks. Experience replay and target networks stabilize learning.
2. **`RL` Implementation Libraries and Tools:**
   - **`Gymnasium (gymnasium.farama.org):`** The maintained fork of `OpenAI Gym`. Your custom environment must subclass `gymnasium.Env` and implement `reset()`, `step()`, `action_space`, and `observation_space`.

- **`Stable-Baselines3`:** Reference implementations of `DQN`, `PPO`, `A2C`. May be useful for validation or as a comparison baseline.
- **`TensorBoard / Weights & Biases`:** For logging training metrics, learning curves, and diagnostic plots.

3. **When RL is Appropriate:**
   - Your problem involves a sequence of decisions where earlier choices affect later possibilities.
   - The quality of a decision can be evaluated through rewards from interactions with the environment.
   - You have a way to simulate the environment or gather interaction data.
   - RL is *not* appropriate when you have large labeled datasets and supervised learning would be simpler and more sample-efficient.

Required Artifacts and Methodology

1. `MDP` **Formulation Document:**

   Before writing any code, produce a written `MDP` specification document covering all five components:
   - *Problem Statement*: What task is the RL agent trying to solve? Why is RL appropriate for your domain compared to supervised learning?
   - *State Space*: What information does the agent observe at each time step? If using neural network features, which features? What is the state dimension?
   - *Action Set*: What actions can the agent take? Are actions discrete or continuous? Enumerate all actions and their interpretations.
   - *Transition Dynamics*: How do actions affect the environment? Describe the transition function: $s_{t+1} = f(s_t, a_t)$. What aspects are deterministic vs. stochastic?
   - *Reward Function*: How does the agent know if it is making progress? Define the reward for each type of transition. What time horizon matters (single-step or long-term cumulative reward)? How will you handle sparse rewards if applicable?
   - *Episode Termination*: When does an episode end? Is there a maximum horizon or terminal condition? How does termination relate to task success?

2. **Environment Implementation (`src/env.py`):**
   - Subclass `gymnasium.Env` and define `action_space` and `observation_space` appropriate to your problem
   - Implement `reset()` to initialize a new episode
   - Implement `step(action)` with complete transition, reward, and termination logic

- Validate the environment by running 100 random-policy episodes. Report: mean episode return, episode length distribution, and action frequency (verify that all actions occur under random policy). These diagnostics confirm correct implementation before training.
- Implement a `render()` method for visualization during debugging and analysis

3. **RL Agent Training (`src/agent.py`):**

   - Choose an appropriate algorithm: `DQN` for discrete action spaces, policy gradient (`PPO`, `A2C`) for continuous or large discrete spaces, or another method suited to your problem.
   - Implement the full training loop with: loss computation, backpropagation, optimization, and periodic evaluation.
   - For `DQN` specifically: experience replay buffer, target network updates, epsilon-greedy exploration with decay, gradient clipping.
   - For policy gradient methods: advantage estimation, entropy regularization, and policy updates.
   - Train for a sufficient number of episodes. Log per-episode metrics: return (cumulative reward), episode length, exploration rate (epsilon for `DQN`).
   - Implement a validation loop: periodically evaluate on a held-out validation set or replay buffer without exploration. Report smoothed validation return.
   - Save the best checkpoint (highest validation return) as `checkpoints/agent_best.pt`

4. **Hyperparameter Search (3+ Configurations):**

   Train at least three agent configurations varying key hyperparameters. Examples:

   - Learning rate: 10^{-5}, 10^{-4}, 10^{-3}
   - Discount factor γ: 0.90, 0.95, 0.99
   - Exploration decay: 0.990, 0.995, 0.999 (for ϵ-greedy)
   - Regularization: no regularization, L_2 weight decay 10^{-5}, L_2 weight decay 10^{-4}

   Report final validation performance for each configuration. Select the best configuration for test-set evaluation.

5. **Evaluation and Comparison:**

   - Evaluate the best agent checkpoint on a held-out test set or test environment. Report the final return and other domain-relevant metrics (success rate, cumulative cost, etc.).
   - Implement baseline strategies: random policy, greedy heuristic (if applicable), or fixed strategies. Evaluate baseline performance on the same test set.
   - Create a comparison table: RL agent vs. baselines. Include metrics from Workshop 1 and Workshop 2 if they measure related quantities.

- Produce trajectory visualizations: show 3–5 example episodes where the agent interacts with the environment, highlighting state transitions, actions taken, and rewards received.

6. **Sample Efficiency Analysis:**
   - Plot learning curves: return vs. number of environment steps (or episodes). Compare against baselines.
   - Analyze how much data (environment interactions) the agent needed to reach a given performance threshold, compared to supervised learning baselines from Workshop 1/2.

Deliverables and Documentation

1. **`RL` Report (`PDF`):**
   - `MDP` specification and problem justification
   - Environment implementation details and random-policy diagnostics
   - Agent algorithm choice and architecture
   - Hyperparameter search results table and training curves for all configurations
   - Test-set evaluation metrics and comparison vs. baselines
   - Sample trajectory visualizations
   - Discussion: did RL outperform supervised learning? What were the key challenges? How did exploration vs. exploitation trade-off manifest?
2. **Code:**
   - `src/env.py` — custom environment implementation
   - `src/agent.py` — agent algorithm (DQN, policy gradient, etc.)
   - `src/baselines.py` — baseline strategies (random, heuristic, etc.)
   - `src/train.py` — training script
   - `src/evaluate.py` — evaluation on test set
   - `checkpoints/agent_best.pt` — best checkpoint
   - `runs/workshop3/` — training logs (TensorBoard/W&B)

Submission Requirements

- The `PDF` report must follow the naming convention: `groupN_workshop3.pdf`
- Submit through the designated course platform by the specified deadline
- Include the `GitHub` repository link with all code and training logs
- All reported metric values must be reproducible by running the submitted code
- RL agents must show measurable learning (validation return increasing over training). If performance does not improve, diagnose the issue and document it in the report.

Grading Note: 40% of the workshop grade will be determined by an in-class quiz administered during the course. This quiz will assess your team's understanding of MDP formulation, agent algorithm design, and reinforcement learning principles. The quiz emphasizes mastery of concepts and the ability to explain your reward function, exploration strategy, and training results.

Important Guidelines and Considerations

- **Validate the Environment First:** Run 100 random-policy episodes and verify the diagnostics make sense. A bug in the reward function or termination logic will silently produce a non-converging agent.
- **Reward Shaping Matters Greatly:** The reward function is your communication to the agent about what to optimize. Sparse rewards (only at episode end) make learning slow; dense rewards (every step) can mislead the agent. Experiment with reward shaping.
- **Exploration is Critical:** If the agent does not explore adequately, it will converge to a suboptimal policy. Monitor exploration metrics (epsilon decay rate, entropy of policy) during training.
- **RL is Sample-Inefficient:** Supervised learning typically needs far fewer data examples than RL needs environment interactions. Ensure your experiment has realistic data budgets.
- **Stabilize Training:** If training is unstable (high variance across runs), apply techniques like gradient clipping, learning rate scheduling, and value normalization.
- **Compare Fairly:** When comparing RL to supervised methods, ensure both use the same amount of computation/data. RL agents need many environment interactions; supervised models need labeled examples. The comparison should be on total sample complexity, not cherry-picked metrics.

Good luck, and embrace the challenge of reinforcement learning — watching an agent discover strategies through trial and error, progressing from random actions to coherent decision-making, is one of the most rewarding experiences in machine learning!